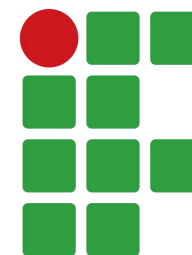


PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco

Agenda da aula

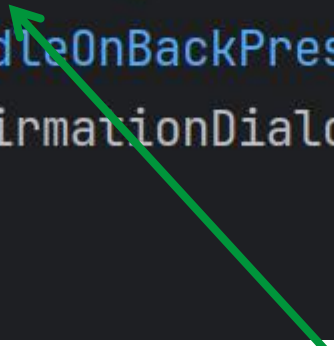
- Continuação do aplicativo exemplo de Login:
 - Tratamento do evento do botão de voltar.
 - Criando AlertDialog.
 - Criando e tratando eventos em menus.
 - Criando intents implícitas.

Tratando o evento do botão de voltar

```
class SecondActivity : AppCompatActivity() {  
  
    4 Usages  
    private lateinit var binding: ActivitySecondBinding  
  
    1 Usage  
    ➔ private val callback = object : OnBackPressedCallback(enabled = true){  
        override fun handleOnBackPressed() {  
            showExitConfirmationDialog()  
        }  
    }  
}
```

Tratando o evento do botão de voltar

```
class SecondActivity : AppCompatActivity() {  
  
    4 Usages  
    private lateinit var binding: ActivitySecondBinding  
  
    1 Usage  
    private val callback = object : OnBackPressedCallback(enabled = true){  
        override fun handleOnBackPressed() {  
            showExitConfirmationDialog()  
        }  
    }  
}
```



callback é criado para interceptar o evento de voltar (Back) do Android e definir o que deve acontecer quando o usuário pressioná-lo.

O object cria um objeto anônimo de uma classe que herda de OnBackPressedCallback.

Tratando o evento do botão de voltar

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    binding = ActivitySecondBinding.inflate(layoutInflater)  
    setContentView(binding.root)  
  
    val user = "${"usuário:"} " +  
        "${intent.getStringExtra(name = MainActivity.EXTRA_USER)}"  
    val dataHour = "${"logado em:"} " +  
        "${intent.getStringExtra(name = MainActivity.EXTRA_DATA_HOUR)}"  
  
    binding.tvUser.text = user  
    binding.tvDateHour.text = dataHour  
  
    onBackPressedDispatcher.addCallback(owner = this, onBackPressedCallback = callback)  
}
```

O `OnBackPressedDispatcher` é o componente responsável por receber o evento de voltar (Back) e encaminhá-lo para os `OnBackPressedCallback` registrados.

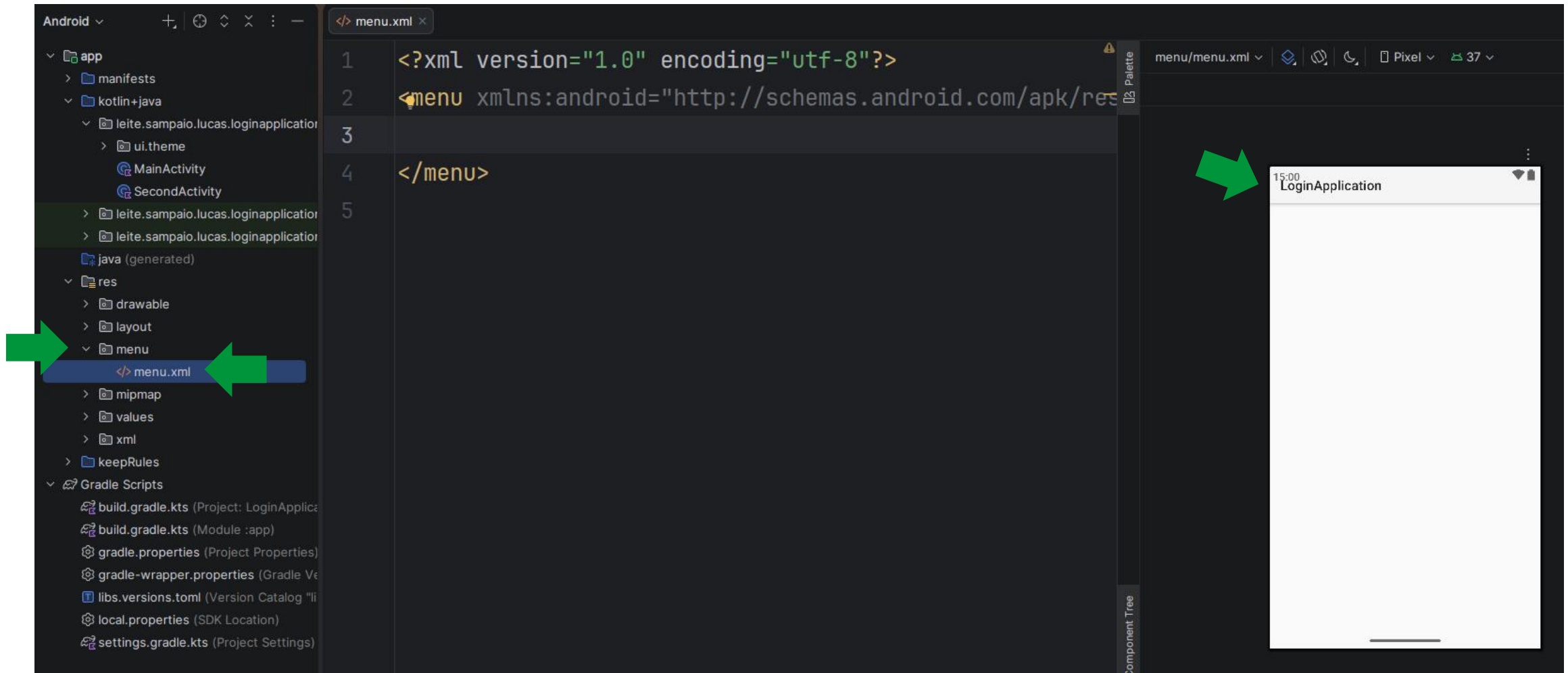
Implementando showExitConfirmationDialog()

```
import androidx.appcompat.app.AlertDialog
```

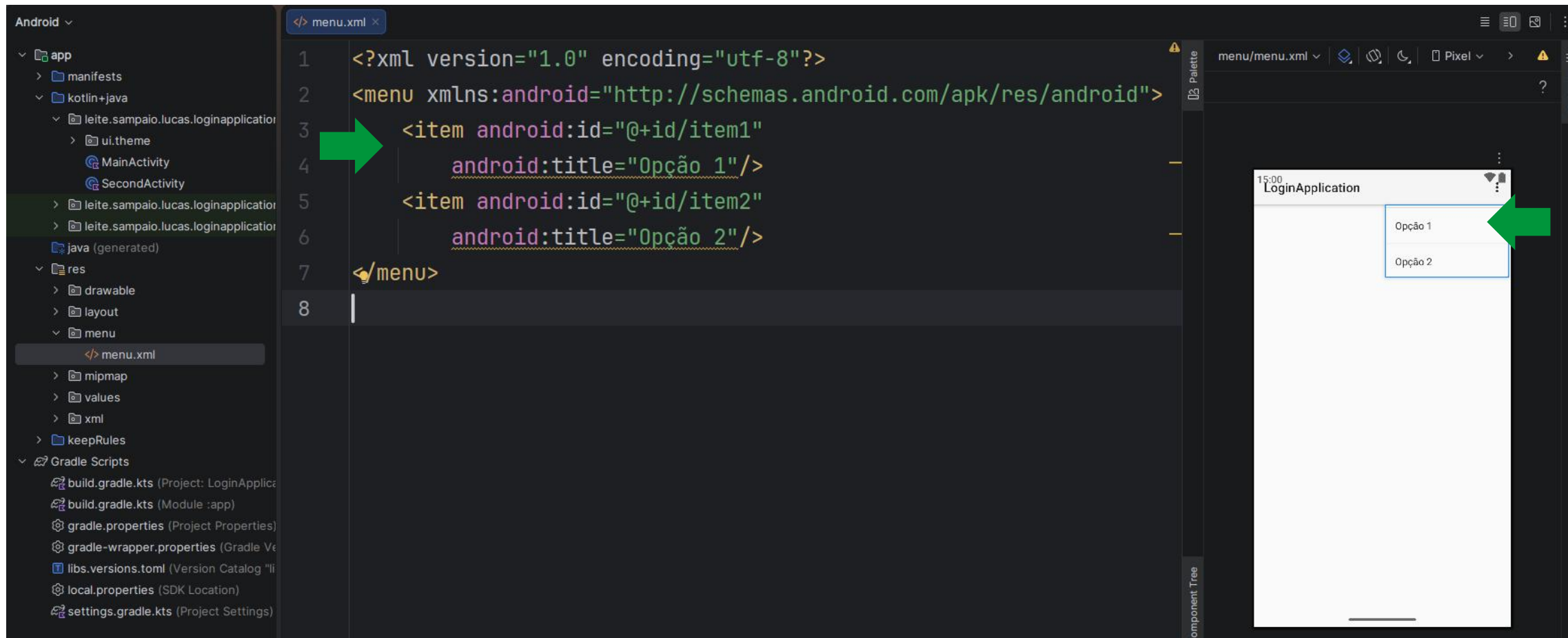
```
private fun showExitConfirmationDialog(){  
    AlertDialog.Builder(context = this)  
        .setTitle("Sair")  
        .setMessage("Deseja realmente sair?")  
        .setPositiveButton(text = "Sim") { _, _ ->  
            finish()  
        }  
        .setNegativeButton(text = "Não", listener = null)  
        .show()  
}
```

Criando um menu

- criar diretório res/menu
 - new → Directory
- criar arquivo menu.xml
 - new → Menu Resource File



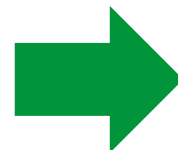
Criando um menu



Associando o layout do menu à Activity

```
override fun onCreateOptionsMenu(menu: Menu?): Boolean {  
    menuInflater.inflate(menuRes = R.menu.menu, menu)  
    return true  
}
```

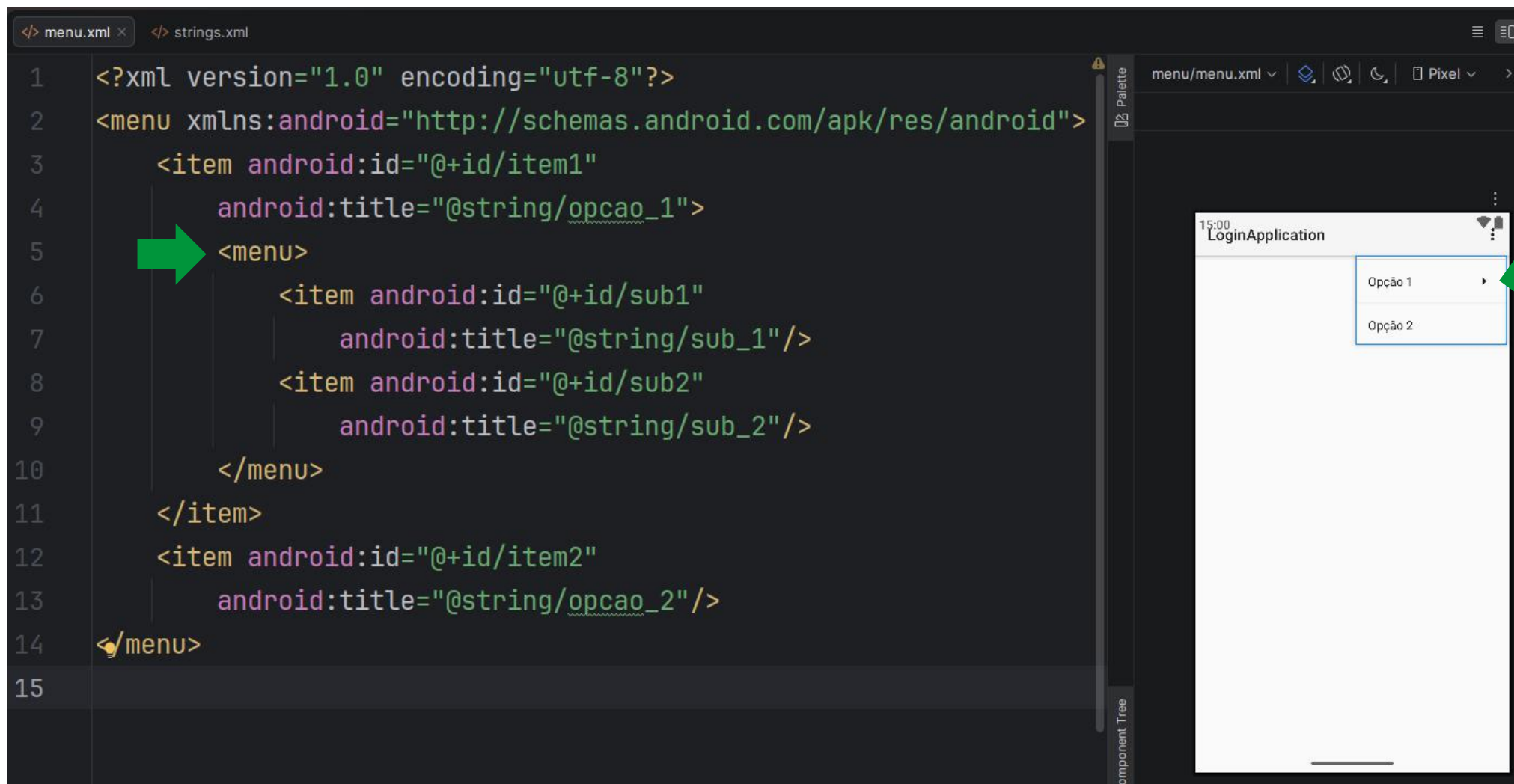
Executando a aplicação



Modificando o título no ActionBar

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    binding = ActivitySecondBinding.inflate(layoutInflater)  
    setContentView(binding.root)  
  
    ➡ setTitle("IFPE - Campus Belo Jardim")  
  
    val user = "${"usuário:"} " +  
        "${intent.getStringExtra(name = MainActivity.EXTRA_USER)}"  
    val dataHour = "${"logado em:"} " +  
        "${intent.getStringExtra(name = MainActivity.EXTRA_DATA_HOUR)}"  
  
    binding.tvUser.text = user  
    binding.tvDateHour.text = dataHour  
  
    onBackPressedDispatcher.addCallback(owner = this, onBackPressedCallback = callback)  
}
```

Criando subitens em um menu



The image shows the Android Studio interface with two panels. The left panel displays the XML code for a menu file named `menu.xml`. The code defines a menu with two main items. The first item, `Opção 1`, has a sub-menu containing two sub-items, `sub_1` and `sub_2`. The second item is `Opção 2`. A green arrow points to the `<menu>` tag within the first item's definition.

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <menu xmlns:android="http://schemas.android.com/apk/res/android">
3     <item android:id="@+id/item1"
4         android:title="@string/opcao_1">
5         <menu>
6             <item android:id="@+id/sub1"
7                 android:title="@string/sub_1"/>
8             <item android:id="@+id/sub2"
9                 android:title="@string/sub_2"/>
10        </menu>
11    </item>
12    <item android:id="@+id/item2"
13        android:title="@string/opcao_2"/>
14 </menu>
```

The right panel shows a preview of the menu UI on a device named "Pixel". The menu is titled "LoginApplication" and contains two items: "Opção 1" and "Opção 2". A green arrow points to the right side of the "Opção 1" item, indicating the sub-menu.

Tratando eventos de clique do menu



```
override fun onOptionsItemSelected(item: MenuItem): Boolean {  
    return when(item.itemId){  
        R.id.item1 -> {  
            Toast.makeText(context = this, text = "Item 1 foi selecionada", duration = Toast.LENGTH_SHORT).show()  
            true  
        }  
        R.id.sub1 -> {  
            Toast.makeText(context = this, text = "Subitem 1 foi selecionada", duration = Toast.LENGTH_SHORT).show()  
            true  
        }  
        R.id.sub2 -> {  
            Toast.makeText(context = this, text = "Subitem 2 foi selecionada", duration = Toast.LENGTH_SHORT).show()  
            true  
        }  
        R.id.item2 -> {  
            Toast.makeText(context = this, text = "Item 2 foi selecionada", duration = Toast.LENGTH_SHORT).show()  
            true  
        }  
        else -> {super.onOptionsItemSelected(item)}  
    }  
}
```

Retomando Intents

- Intents Explícitas: são usadas quando o componente de destino é conhecido. Você especifica explicitamente qual componente deve ser iniciado, geralmente dentro do próprio aplicativo.
- Intents Implícitas: são usadas quando o componente de destino não é especificado. Você informa uma ação e, opcionalmente, dados ou um tipo de conteúdo, e o Android determina qual componente pode atender à solicitação.

Retomando Intents

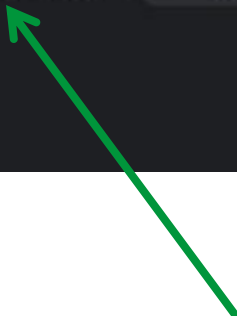
- Em uma Intent implícita, a ação (action) e a URI (Uniform Resource Identifier) são elementos importantes para indicar o que deve ser realizado e sobre qual recurso ou tipo de dado a ação será executada.
- A ação (action) é uma String que descreve a operação que se deseja realizar. Alguns exemplos são:
 - ACTION_VIEW → visualizar ou abrir um recurso.
 - ACTION_SEND → enviar ou compartilhar dados com outro componente.
 - ACTION_DIAL → abrir o discador com um número de telefone informado.
- A URI (Uri) identifica o recurso sobre o qual a ação será realizada, como uma página da Web, um número de telefone ou uma localização.

Definidos Intents implícitas

```
R.id.sub1 -> {  
    val url = "https://portal.ifpe.edu.br/belo-jardim/"  
    val intent = Intent(action = Intent.ACTION_VIEW , uri = url.toUri())  
    startActivity(intent)  
    true  
}  
  
R.id.sub2 -> {  
    val url = "https://play.kotlinlang.org/"  
    val intent = Intent(action = Intent.ACTION_VIEW , uri = url.toUri())  
    startActivity(intent)  
    true  
}
```

Definidos Intents implícitas

```
R.id.item2 -> {  
    val text = "Testando opção de compartilhamento de texto com outras aplicações..."  
    val intent = Intent(action = Intent.ACTION_SEND)  
    intent.type = "text/plain"  
    intent.putExtra(name = Intent.EXTRA_TEXT, value = text)  
    startActivity(Intent.createChooser(target = intent, title = "Compartilhar via..."))  
    true  
}
```



O método `createChooser()` é utilizado para criar um diálogo de escolha no Android, onde o usuário pode selecionar entre diferentes aplicativos que podem lidar com a intent implícita especificada.

Definidos Intents implícitas

```
R.id.item2 -> {  
    val text = "Testando opção de compartilhamento de texto com outras aplicações..."  
    val intent = Intent(action = Intent.ACTION_SEND)  
    intent.type = "text/plain"  
    intent.putExtra(name = Intent.EXTRA_TEXT, value = text)  
    startActivity(Intent.createChooser(target = intent, title = "Compartilhar via..."))  
    true  
}
```

Title é uma sugestão de título para o Chooser (seletor de aplicativos), mas o Android pode decidir não exibi-lo ou pode usar a própria interface do sistema dependendo da versão do Android e do tipo de Intent.

Exercício

- Crie um terceiro item no menu para abrir o aplicativo Google Maps, utilizando uma Intent implícita e informando as coordenadas do IFPE - Campus Belo Jardim.



Exercício

- Crie um terceiro item no menu para abrir o aplicativo Google Maps, utilizando uma Intent implícita e informando as coordenadas do IFPE - Campus Belo Jardim.

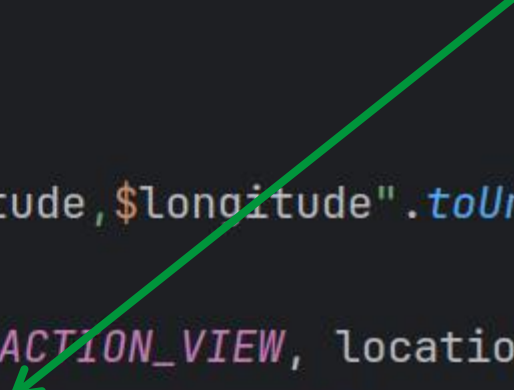
```
R.id.item3 -> {  
    val latitude = -8.3200929  
    val longitude = -36.4182826  
  
    val locationUri = "geo:0,0?q=$latitude,$longitude".toUri()  
  
    val intent = Intent(action = Intent.ACTION_VIEW, locationUri)  
    intent.setPackage("com.google.android.apps.maps")  
  
    startActivity(intent)  
    true  
}
```

Exercício

- Crie um terceiro item no menu para abrir o aplicativo Google Maps, utilizando uma Intent implícita e informando as coordenadas do IFPE - Campus Belo Jardim.

```
R.id.item3 -> {  
    val latitude = -8.3200929  
    val longitude = -36.4182826  
  
    val locationUri = "geo:0,0?q=$latitude,$longitude".toUri()  
  
    val intent = Intent(action = Intent.ACTION_VIEW, locationUri)  
    intent.setPackage("com.google.android.apps.maps")  
  
    startActivity(intent)  
    true  
}
```

E se o Google Maps não estiver instalado no smartphone/emulador?



Exercício

- Crie um terceiro item no menu para abrir o aplicativo Google Maps, utilizando uma Intent implícita e informando as coordenadas do IFPE - Campus Belo Jardim.

```
R.id.item3 -> {  
    val latitude = -8.3200929  
    val longitude = -36.4182826  
  
    val locationUri = "geo:0,0?q=$latitude,$longitude".toUri()  
  
    val intent = Intent(action = Intent.ACTION_VIEW, locationUri)  
    intent.setPackage("com.google.android.apps.maps")  
  
    startActivity(intent)  
    true  
}
```

E se o Google Maps não estiver instalado no smartphone/emulador?

startActivity(intent) pode lançar:
ActivityNotFoundException



Soluções: deixar o Android escolher o aplicativo

```
R.id.item3 -> {  
    val latitude = -8.3200929  
    val longitude = -36.4182826  
  
    val locationUri = "geo:0,0?q=$latitude,$longitude".toUri()  
  
    val intent = Intent(action = Intent.ACTION_VIEW, locationUri)  
  
    startActivity(intent)  
    true  
}
```

Soluções: deixar o Android escolher o aplicativo

```
R.id.item3 -> {  
    val latitude = -8.3200929  
    val longitude = -36.4182826  
  
    val locationUri = "geo:0,0?q=$latitude,$longitude".toUri()  
  
    val intent = Intent(action = Intent.ACTION_VIEW, locationUri)  
  
    startActivity(intent)  
    true  
}
```

E se não houver nenhum aplicativo capaz de tratar essa URI?

Soluções: tratando a ausência de um aplicativo compatível

```
R.id.item3 -> {  
    val latitude = -8.3200929  
    val longitude = -36.4182826  
  
    val locationUri = "geo:0,0?q=$latitude,$longitude".toUri()  
  
    val intent = Intent(action = Intent.ACTION_VIEW, locationUri)  
  
    if (intent.resolveActivity(pm = packageManager) != null) {  
        startActivity(intent)  
    } else {  
        Toast.makeText(context = this,  
            text = "Nenhum aplicativo de mapas instalado.",  
            duration = Toast.LENGTH_LONG).show()  
    }  
    true  
}
```

Soluções: tratando a ausência de um aplicativo compatível

- Quando a aplicação precisa verificar se existe um componente capaz de atender a uma Intent, por exemplo utilizando `resolveActivity()`, é necessário declarar no `<queries>` o tipo de componente que será consultado.
- O elemento `<queries>` no `AndroidManifest.xml` permite declarar quais aplicativos ou tipos de componentes a aplicação precisa consultar no dispositivo. Essa declaração está relacionada às restrições de visibilidade de pacotes introduzidas no Android 11 (API 30).

```
<queries>
  <intent>
    <action android:name="android.intent.action.VIEW" />
    <data android:scheme="geo" />
  </intent>
</queries>
```

Soluções: e se o requisito for especificamente Google Maps?

```
R.id.item3 -> {  
    val latitude = -8.3200929  
    val longitude = -36.4182826  
  
    val locationUri = "geo:0,0?q=$latitude,$longitude".toUri()  
  
    val intent = Intent(action = Intent.ACTION_VIEW, locationUri)  
    intent.setPackage("com.google.android.apps.maps")  
  
    if (intent.resolveActivity(pm = packageManager) != null) {  
        startActivity(intent)  
    } else {  
        Toast.makeText(context = this,  
            text = "Nenhum aplicativo de mapas instalado.",  
            duration = Toast.LENGTH_LONG).show()  
    }  
    true  
}
```

Soluções: e se o requisito for especificamente Google Maps?

```
<queries>
  <package android:name="com.google.android.apps.maps" />
</queries>
```

Exercício

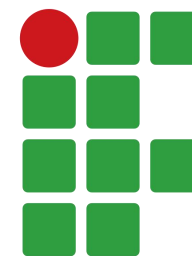
1. Realize uma revisão do código, identificando possíveis problemas ou oportunidades de melhoria relacionados às boas práticas de desenvolvimento Android.
2. Na sua análise, verifique, entre outros aspectos:
 - Se existem strings visíveis ao usuário que deveriam estar no strings.xml;
 - Se os valores utilizados no código poderiam ser melhor organizados ou declarados como constantes;
 - Se existe algum problema caso o Google Maps não esteja instalado;
 - Se há alguma melhoria relacionada à legibilidade e organização do código.

Dúvidas



PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco